

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP HỆ CỬ NHÂN
KỲ 20252

Thông tin về sinh viên

Họ và tên sinh viên:	Đinh Đức Hiếu	MSSV:	20225314
Điện thoại liên lạc:	856596998	Lớp:	Kỹ thuật máy tính 02 - K67
Email:	Hieu.DD225314@sis.hust.edu.vn	Mã lớp:	IT2-02

Thông tin giáo viên hướng dẫn

Họ và tên GVHD:	Nguyễn Thị Thanh Nga
Đồ án được thực hiện tại:	Trường Công nghệ thông tin và Truyền thông
Thời gian làm ĐATN:	Từ ngày 23/2/2026 đến ngày 30/6/2026

1. Tên đề tài:

Thiết kế và xây dựng hệ thống cảnh báo sạt lở đất thời gian thực

2. Lĩnh vực đề tài:

- Lựa chọn 1: **AIoT**
- Lựa chọn 2:
- Lựa chọn 3:
- Nếu lĩnh vực không nằm trong danh sách có sẵn, giáo viên hướng dẫn có thể đề xuất:

3. Mục tiêu của ĐATN:

3.1. Kiến thức sinh viên thu thập được:

- Nắm vững kiến thức về cảm biến, vi điều khiển và kiến trúc hệ thống nhúng IoT ứng dụng trong quan trắc môi trường.
- Phát triển các thành phần của một hệ thống ứng dụng trên nền Web hiện đại (frontend, backend, database, realtime, websocket, AI).
- Cách phân tích các yêu cầu của hệ thống, luồng hoạt động và các vấn đề liên quan.

3.2. Công nghệ sinh viên thu thập được:

- Công nghệ IoT:
 - + Sử dụng vi điều khiển ESP32 lập trình bằng C++ để thu thập dữ liệu từ cảm biến.

- + Áp dụng thuật toán băm SHA-256 nhằm đảm bảo tính toàn vẹn và bảo mật của dữ liệu truyền đi.
- + Sử dụng công nghệ LoRa để truyền dữ liệu giữa các node không có kết nối Internet và Gateway.
- + Áp dụng giao thức MQTT để truyền dữ liệu thời gian thực từ Gateway tới Backend Server.
- Công nghệ Web Frontend:
 - + Sử dụng Next.js (App Router), React, và TypeScript để xây dựng giao diện người dùng.
 - + Sử dụng Tailwind CSS kết hợp với các component UI từ shadcn/ui để phát triển giao diện nhanh và nhất quán.
 - + Recharts cho biểu đồ dashboard/sensor.
 - + Tích hợp thư viện biểu đồ (Recharts) để hiển thị dữ liệu cảm biến..
 - + Sử dụng Leaflet để hiển thị bản đồ và vị trí thiết bị.
 - + Sử dụng Socket.IO Client để cập nhật dữ liệu thời gian thực.
- Công nghệ phát triển Backend:
 - + Xây dựng hệ thống Backend bằng Node.js và Express theo mô hình REST API.
 - + Sử dụng PostgreSQL để lưu trữ dữ liệu và Redis cho caching và quản lý session.
 - + Tích hợp MQTT client để nhận dữ liệu từ Gateway.
 - + Sử dụng Socket.IO để truyền dữ liệu realtime tới frontend.
 - + Áp dụng JWT và bcrypt để xác thực và bảo mật người dùng.
 - + Sử dụng node-cron để giám sát trạng thái thiết bị theo lịch.
- Công nghệ Trí tuệ nhân tạo (AI/ML):
 - + Sử dụng Python kết hợp với thư viện scikit-learn.
 - + Áp dụng mô hình Random Forest để phân loại mức độ cảnh báo (SAFE, WARNING, DANGER).
- Dịch vụ bên thứ ba và tích hợp:
 - + Sử dụng SendGrid để gửi email cảnh báo.
 - + Sử dụng bản đồ nền từ OpenStreetMap.
- Triển khai hệ thống lên cloud:
 - + Sử dụng Neon PostgreSQL để triển khai cơ sở dữ liệu.
 - + Backend Server được triển khai trên Render.
 - + Frontend được triển khai trên Vercel.
 - + Sử dụng HiveMQ để triển khai broker MQTT.

3.3. Kỹ năng sinh viên phát triển được:

- Thiết kế và xây dựng hệ thống IoT: thu thập dữ liệu từ cảm biến, truyền dữ liệu qua LoRa và xử lý tại Gateway.
- Xây dựng hệ thống Backend: phát triển REST API, xử lý dữ liệu thời gian thực thông qua MQTT và Socket.IO.
- Phát triển giao diện Web: xây dựng dashboard hiển thị dữ liệu cảm biến, biểu đồ và bản đồ trực quan.
- Làm việc với cơ sở dữ liệu: thiết kế và quản lý dữ liệu bằng PostgreSQL, tối ưu truy vấn và lưu trữ.
- Triển khai hệ thống lên môi trường cloud: đưa hệ thống vào hoạt động thực tế, đảm bảo khả năng truy cập từ xa và hoạt động ổn định.
- Xây dựng và tích hợp mô hình AI: sử dụng mô hình học máy để phân loại mức độ cảnh báo (SAFE, WARNING, DANGER).
- Xử lý hệ thống thời gian thực: đồng bộ dữ liệu giữa các thành phần IoT, Backend và Frontend.
- Xử lý lỗi hệ thống: xác định và khắc phục lỗi giữa các thành phần như thiết bị, mạng truyền thông và server.
- Phân tích và giải quyết vấn đề: đưa ra phương án tối ưu trong quá trình thiết kế và triển khai hệ thống.

3.4. Sản phẩm kỳ vọng:

- Xây dựng hệ thống giám sát và cảnh báo sạt lở dựa trên nền tảng IoT, cho phép thu thập và phân tích dữ liệu cảm biến theo thời gian thực.

- Mô hình phần cứng IoT:
 - + Hệ thống các thiết bị Node (sử dụng vi điều khiển ESP32, module giao tiếp LoRa và các cảm biến đo lường gồm độ ẩm đất, lượng mưa, góc, rung).
 - + Thiết bị Gateway (sử dụng vi điều khiển ESP32 và module giao tiếp LoRa) làm nhiệm vụ thu thập dữ liệu từ các Node và truyền tải lên máy chủ.
- Hệ thống Backend Server & API:
 - + Máy chủ xử lý lưu trữ dữ liệu và gọi API module AI theo thời gian thực từ Gateway thông qua giao thức MQTT.
 - + Hệ thống RESTful API phục vụ việc quản lý thiết bị, lưu trữ dữ liệu cảm biến và xác thực người dùng.
- Ứng dụng Web Quản trị và Giám sát:
 - + Dashboard trực quan: Hiển thị dữ liệu cảm biến theo thời gian thực (biểu đồ, trạng thái hoạt động của thiết bị).
 - + Bản đồ định vị: Tích hợp bản đồ hiển thị vị trí các node và phân vùng cảnh báo theo khu vực địa lý.
 - + Hệ thống cảnh báo: Gửi thông báo ngay lập tức tới người dùng khi hệ thống phát hiện dấu hiệu bất thường.
 - + Giao diện cho phép cấu hình, thêm/sửa/xóa các thiết bị phần cứng và tra cứu lịch sử dữ liệu.
- Module Trí tuệ nhân tạo:
 - + Mô hình học máy được huấn luyện để phân tích chuỗi dữ liệu cảm biến và phân loại mức độ rủi ro (SAFE, WARNING, DANGER).
 - + Thuật toán hỗ trợ dự báo và phát hiện sớm các nguy cơ tiềm ẩn.
- Hạ tầng triển khai:
 - + Toàn bộ hệ thống (Web, Backend, Database) được đóng gói và vận hành ổn định trên môi trường Điện toán đám mây (Cloud), đảm bảo khả năng truy cập từ xa và hoạt động liên tục 24/7 qua Internet.

3.5. Vấn đề thực tiễn đề án giải quyết:

- Trong thực tế, các hiện tượng như sạt lở đất thường xảy ra bất ngờ, gây thiệt hại lớn về người và tài sản. Việc giám sát hiện nay chủ yếu dựa vào quan sát thủ công hoặc các hệ thống rời rạc, thiếu tính liên tục và khó phát hiện sớm các dấu hiệu nguy hiểm.
- Các hệ thống giám sát truyền thống còn tồn tại nhiều hạn chế:
 - + Không cung cấp dữ liệu theo thời gian thực.
 - + Khó triển khai tại các khu vực địa hình phức tạp, thiếu kết nối Internet.
 - + Thiếu khả năng phân tích và đưa ra cảnh báo tự động.
- Đề án đề xuất xây dựng một hệ thống giám sát dựa trên công nghệ IoT, cho phép:
 - + Thu thập dữ liệu liên tục từ các cảm biến môi trường.
 - + Truyền dữ liệu từ các khu vực không có Internet thông qua công nghệ LoRa.
 - + Phân tích dữ liệu và đưa ra cảnh báo theo thời gian thực thông qua mô hình AI.
 - + Hiển thị và giám sát dữ liệu từ xa thông qua ứng dụng Web.
- Hệ thống góp phần hỗ trợ phát hiện sớm các nguy cơ, giúp người dùng có thể đưa ra các biện pháp phòng tránh kịp thời, giảm thiểu thiệt hại trong thực tế.

4. Các nội dung sẽ thực hiện và kế hoạch triển khai:

Lưu ý: khối lượng yêu cầu đối với đề án tốt nghiệp hệ cử nhân là 6(0-0-12-12), i.e. 12 tiết làm việc/tuần trong 17

Nội dung 1: Tìm hiểu tổng quan về bài toán,

từ Tuần 1 đến Tuần 2

Chi tiết:

- Khảo sát thực trạng các phương pháp giám sát sạt lở hiện nay:
 - + Các phương pháp thủ công (quan sát trực tiếp, đo đạc định kỳ).

- + Các hệ thống giám sát sử dụng cảm biến trong nghiên cứu và thực tế.
- Phân tích các hạn chế của hệ thống hiện tại:
 - + Không cung cấp dữ liệu liên tục theo thời gian thực.
 - + Khó triển khai tại các khu vực địa hình phức tạp, thiếu kết nối Internet.
 - + Thiếu khả năng cảnh báo sớm và tự động.
- Nghiên cứu các công nghệ áp dụng trong hệ thống:
 - + Công nghệ IoT (ESP32, cảm biến môi trường).
 - + Công nghệ truyền dữ liệu tầm xa LoRa.
 - + Giao thức truyền thông MQTT.
 - + Công nghệ Web (Frontend/Backend) phục vụ giám sát từ xa.
 - + Ứng dụng AI trong phân tích và cảnh báo sạt lở.
- Xác định yêu cầu chức năng:
 - + Thu thập và lưu trữ dữ liệu từ cảm biến theo thời gian thực.
 - + Hiển thị dữ liệu và trạng thái thiết bị trên dashboard.
 - + Hiển thị vị trí thiết bị trên bản đồ.
 - + Cảnh báo khi phát hiện nguy cơ sạt lở.
- Xác định yêu cầu phi chức năng:
 - + Đảm bảo hệ thống hoạt động ổn định, liên tục.
 - + Đảm bảo độ trễ thấp trong việc truyền và xử lý dữ liệu.
 - + Khả năng mở rộng khi tăng số lượng thiết bị.

Nội dung 2: Tìm hiểu tổng quan về công nghệ liên quan,

từ Tuần

3

đến Tuần

4

Chi tiết:

- Công nghệ IoT:
 - + Trong hệ thống, vi điều khiển ESP32 được sử dụng để đọc dữ liệu từ cảm biến và gửi về Gateway.
 - + Công nghệ truyền thông LoRa là công nghệ truyền dữ liệu tầm xa, tiêu thụ năng lượng thấp, phù hợp với các khu vực không có Internet.
 - + Giao thức MQTT là giao thức publish/subscribe nhẹ, phù hợp cho hệ thống IoT. Giúp truyền dữ liệu từ Gateway đến Backend Server theo thời gian thực.
- Công nghệ Backend:
 - + Sử dụng Node.js và Express để xây dựng REST API.
 - + Hỗ trợ xử lý dữ liệu, quản lý thiết bị và cung cấp dữ liệu cho frontend.
- Cơ sở dữ liệu PostgreSQL:
 - + Được sử dụng để lưu trữ dữ liệu cảm biến, thông tin thiết bị và lịch sử cảnh báo.
 - + Đảm bảo tính toàn vẹn và khả năng truy vấn dữ liệu hiệu quả.
- Công nghệ Frontend:
 - + Sử dụng React và Next.js để xây dựng giao diện người dùng.
 - + Hỗ trợ hiển thị dữ liệu dưới dạng biểu đồ, bản đồ và cập nhật theo thời gian thực.
- Công nghệ realtime (Socket.IO):
 - + Cho phép truyền dữ liệu tức thời giữa Backend và Frontend.
 - + Đảm bảo người dùng nhận được cảnh báo ngay khi có sự cố.
- Công nghệ Trí tuệ nhân tạo:
 - + Sử dụng scikit-learn để xây dựng mô hình học máy.
 - + Áp dụng mô hình Random Forest để phân loại mức độ cảnh báo dựa trên dữ liệu cảm biến.

Nội dung 3: Phân tích thiết kế,

từ Tuần

5

đến Tuần

8

Chi tiết:

- Thiết kế kiến trúc tổng thể:
 - + Hệ thống được thiết kế theo mô hình Client–Server kết hợp với kiến trúc IoT nhiều tầng, bao gồm: Tầng thu

thập dữ liệu (IoT Node – ESP32 + cảm biến + Lora), Tầng truyền dữ liệu (Gateway – ESP32 + Lora), Tầng xử lý (Backend Server), Tầng hiển thị (Web Frontend).

+ Backend được xây dựng bằng Node.js và Express theo mô hình RESTful API, đóng vai trò trung tâm xử lý và điều phối dữ liệu.

- Thiết kế cơ sở dữ liệu:

+ Hệ thống sử dụng PostgreSQL để lưu trữ dữ liệu.

+ Các bảng chính gồm Device, Node, SensorDataHistory, Alert, User.

- Thiết kế luồng dữ liệu:

+ Luồng xử lý dữ liệu trong hệ thống:

1. Cảm biến thu thập dữ liệu môi trường.

2. Node gửi dữ liệu qua LoRa đến Gateway.

3. Gateway so sánh ngưỡng cảnh báo và gửi dữ liệu lên hệ thống thông qua giao thức MQTT.

4. Backend nhận dữ liệu, xử lý và lưu vào cơ sở dữ liệu.

5. Mô hình AI phân tích dữ liệu và xác định mức cảnh báo.

6. Backend gửi dữ liệu và cảnh báo realtime tới frontend.

- Thiết kế API và giao tiếp hệ thống:

+ Backend cung cấp các RESTful API: API quản lý tài khoản, API quản lý thiết bị, API truy vấn dữ liệu cảm biến, API quản lý cảnh báo.

+ Sử dụng Socket.IO để truyền dữ liệu realtime từ Backend đến Frontend.

- Thiết kế giao diện người dùng:

+ Giao diện Web được thiết kế bao gồm: Dashboard hiển thị tổng quan dữ liệu toàn hệ thống, bản đồ hiển thị vị trí các thiết bị, trang quản lý các cảnh báo, trang quản lý thiết bị, trang hiển thị lịch sử dữ liệu và trang quản lý tài khoản.

+ Giao diện được tối ưu để hỗ trợ nhiều thiết bị và cập nhật dữ liệu theo thời gian thực.

Nội dung 4: Xây dựng chương trình,

từ Tuần

9

đến Tuần

15

Chi tiết:

- Xây dựng hệ thống IoT:

+ Lập trình vi điều khiển ESP32 bằng C++ để thu thập dữ liệu từ cảm biến.

- + Xây dựng cơ chế mã hóa dữ liệu bằng SHA-256, đóng gói và truyền dữ liệu qua LoRa từ Node đến Gateway.
- + Tại Gateway, kiểm tra các giá trị cảm biến nhằm xác định điều kiện kích hoạt các mức cảnh báo, sau đó chuyển tiếp dữ liệu lên hệ thống server thông qua MQTT Broker.
- Xây dựng Backend:
 - + Triển khai các API bằng Node.js và Express: API quản lý thiết bị và dữ liệu cảm biến, API cung cấp dữ liệu cho Dashboard và API cho quản lý các cảnh báo.
 - + Xây dựng module xử lý dữ liệu: Kiểm tra và xác thực dữ liệu đầu vào., lưu trữ dữ liệu vào PostgreSQL, kích hoạt cảnh báo nếu có trong gói tin nhận được từ Gateway.
 - + Xây dựng cơ chế realtime sử dụng Socket.IO để truyền dữ liệu cảm biến và cảnh báo đến phía Frontend.
 - + Tích hợp dịch vụ email SendGrid để gửi thông báo cảnh báo đến người quản lý theo khu vực.
- Xây dựng Frontend:
 - + Phát triển dashboard hiển thị: biểu đồ dữ liệu cảm biến theo thời gian, trạng thái hoạt động của thiết bị, tổng quan các cảnh báo trong hệ thống.
 - + Xây dựng trang quản lý cảnh báo: hiển thị danh sách và chi tiết các cảnh báo, cho phép cập nhật trạng thái cảnh báo (đã xác nhận, đã xử lý).
 - + Xây dựng trang Bản đồ: tích hợp bản đồ từ OpenStreetMap, hiển thị vị trí các Node và Gateway, sử dụng heatmap để thể hiện mức độ cảnh báo tại các khu vực.
 - + Xây dựng các trang chức năng:
 - Trang quản lý thiết bị: hỗ trợ các thao tác CRUD cho Node và Gateway.
 - Trang lịch sử hệ thống: tra cứu dữ liệu cảm biến và cảnh báo theo thời gian.
 - Trang quản lý tài khoản: chỉnh sửa thông tin và phân quyền quản lý theo khu vực.
 - + Xây dựng cơ chế realtime: nhận dữ liệu từ server thông qua Socket.IO, cập nhật giao diện ngay khi có dữ liệu mới.
- Tích hợp mô hình AI:
 - + Chuẩn bị tập dữ liệu lưu trữ trong bảng dataset của hệ thống: dữ liệu thực tế thu thập từ cảm biến (khoảng 10–20%), dữ liệu giả lập được tạo bằng script (khoảng 80%).
 - + Huấn luyện mô hình bằng thư viện scikit-learn, sử dụng thuật toán Random Forest để dự đoán mức cảnh báo.
 - + Xây dựng một service riêng cho AI, giao tiếp với Backend chính thông qua API.
 - + Triển khai service AI: nhận dữ liệu mới từ Gateway thông qua Backend, thực hiện dự đoán mức cảnh báo, trả kết quả về Backend chính để xử lý và hiển thị.

Nội dung 5: Thử nghiệm và đánh giá,

từ Tuần

16

đến Tuần

17

Chi tiết:

- Kiểm tra các chức năng chính của hệ thống:
- + Thu thập và hiển thị dữ liệu cảm biến.

- + Quản lý thiết bị (thêm, sửa, xóa).
- + Hiển thị và xử lý các cảnh báo.
- + Đảm bảo hệ thống hoạt động ổn định khi có nhiều thiết bị Gateway đồng thời gửi dữ liệu tới.
- Kiểm tra các API Backend:
 - + Đảm bảo dữ liệu được xử lý chính xác và trả về đúng định dạng.
 - + Đảm bảo hệ thống hoạt động ổn định khi có nhiều request đồng thời.
- Thử nghiệm hệ thống IoT:
 - + Kiểm tra khả năng thu thập dữ liệu của các node ESP32 từ cảm biến.
 - + Đánh giá độ ổn định của việc truyền dữ liệu qua LoRa trong điều kiện thực tế.
 - + Kiểm tra việc gửi dữ liệu từ Gateway lên server thông qua MQTT.
 - + Kiểm tra tính ổn định khi 1 Gateway nhận được nhiều gói dữ liệu từ các Node khác nhau.
- Thử nghiệm hệ thống realtime:
 - + Kiểm tra việc truyền dữ liệu từ Backend đến Frontend thông qua Socket.IO.
 - + Đánh giá độ trễ khi dữ liệu được cập nhật trên dashboard.
 - + Đảm bảo cảnh báo được hiển thị ngay khi phát hiện bất thường.
- Thử nghiệm mô hình AI:
 - + Đánh giá độ chính xác của mô hình phân loại (SAFE, WARNING, DANGER).
 - + So sánh kết quả dự đoán với dữ liệu thực tế.
 - + Kiểm tra khả năng tích hợp của service AI với hệ thống Backend.
- Thử nghiệm triển khai hệ thống:
 - + Kiểm tra khả năng hoạt động của hệ thống khi triển khai trên môi trường cloud: Neon (Database), Render (Backend), Vercel (Frontend), HiveMQ (MQTT Broker).
 - + Kết quả cho thấy hệ thống hoạt động ổn định: dữ liệu được truyền qua LoRa và MQTT liên tục, hệ thống hiển thị và cảnh báo theo thời gian thực.

5. Lời cam đoan của sinh viên đã nhận được nhiệm vụ

Em xin cam kết sẽ hoàn thành các nhiệm vụ theo đúng kế hoạch.

Hà Nội, ngày 19 tháng 04 năm 202

Sinh viên

(Ký và ghi rõ họ tên)

Đinh Đức Hiếu

6. Xác nhận của giáo viên hướng dẫn về việc giao nhiệm vụ cho sinh viên

Hà Nội, ngày tháng năm

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

Nguyễn Thị Thanh Nga